

Java 实验报告

软件学院 李宗杭 2311451

实验题目：CDShop

一、设计思路

1、总体架构

CDShop 类：负责初始化 CD 库存、实现进货、销售、租借以及归还方法，创建线程池、启动各功能线程，并维护全局状态

进货线程 RestockThread：定期补充库存，支持紧急补货

销售线程 SellThread：模拟顾客购买 CD 的行为

租借线程 RentThread：模拟顾客租借和归还 CD 的行为

2、存储结构设计

使用两个 **ConcurrentHashMap** 分别存储可租借和可销售的 CD 库存：

rentableCDs：存储可租借 CD，键为 CD 编号，值为是否可借

sellableCDs：存储可销售 CD，键为 CD 编号，值为库存数量

3、线程设计

进货线程 RestockThread：每隔 1 秒执行一次进货操作，将可售 CD 列表的库存补齐到最大库存。如果出现临时缺货的情况，会紧急启动一次进货操作。

销售线程 SellThread：随机选择一种 CD 和购买数量，尝试进行销售。如果库存不足，会随机选择等待或放弃。如果选择等待，会唤醒进货线程进行补货。

租借线程 RentThread：随机选择一种 CD 进行租借。如果该 CD 已经出租，会随机选择等待或放弃。如果成功租借，会随机等待 200-300ms 后归还。

二、同步方式

1、锁机制

程序中使用三个锁来实现同步

文件锁 fileLock: 保证同一时间只有一个线程可以写入文件，避免多个线程同时写入文件导致数据混乱

销售锁 SellLock: 确保销售和补货操作不会同时进行；用于销售线程和进货线程之间的通信

租借锁 RentLock: 确保同一时间只有一个线程能修改租借状态

2、线程间通信

销售线程与进货线程的通信:

- 1、当库存不足时，销售线程通过设置 isEmergency 标志通知进货线程需要紧急补货
- 2、使用 wait()和 notifyAll()实现等待和唤醒机制，进货线程每补货一次便 notifyAll()唤醒所有等待补货的销售线程，随后 wait(1000)固定间隔 1 秒；销售线程判断库存不足时便会 notifyAll()紧急通知进货线程补货，随后 wait()等待进货线程补货后唤醒

租借线程间的协调:

- 1、通过 synchronized 块和 rentLock 确保同一时间只有一个线程能修改租借状态
- 2、使用简单的忙等待机制处理 CD 已被租出的情况

三、源代码及记录文件见附件